

Noel Bryan - CMPSC 132 Final Report

Project Description

I chose the Snakes and Ladders game for my Final Project. I decided to make a two player only game that allows the user to specify both the player's names and the amount of snakes and ladders on the board per game. To make a move, a user simply presses enter and a dice is automatically rolled from one to six for them.

Approach and Logic

I decided to use a class to store the vast majority of my game code, called Game. This game class contains methods to calculate the positions and number of snakes or ladders, display the current board with print statements, and take a user's turn. I also defined a separate class, called Player, to house each individual player's position and name. This approach with classes gives me multiple advantages.

In the takeTurn function in the Game class, I first ask a user to make a roll, then get a random value between 1 and 6. First, I check if that number is over 100, and exit early if it is. Then, I check it against the snakes/ladders dictionary, and act appropriately if it is in either. Finally, I check if the user has won after these changes.

I also have two functions, generateSnakes and generateLadders to build the dictionary of ladders/snakes for the game. These functions loop for a specific amount of time and generate random numbers to start/end each snake. I had to ensure that those numbers were either higher, in the case of ladders, or lower, in the case of snakes.

Finally, I have a displayBoard function as well in my Game class. This uses two while loops to loop over the rows/columns of the board and progressively build a string for each spot on the board. If the spot is a snake, it conveniently marks that with a s->number or l->number in the case of a ladder. This was my hardest function to make, as I had to calculate the spacing between each column.

Data Structures

In my project, I mainly made use of dictionaries and classes. I used dictionaries to store the positions of snakes/ladders, as well as where they lead. This allowed me easy access and an easy way to check if a spot is a snake/ladder, as all I had to do was check if a spot was in the snakes/ladders dictionary. Additionally, I used classes to hold the entire game and both players. This allows for easy extensibility later, if I decide to change something or build off of the Game or Player class.

Challenges Faced

My hardest challenge was building the displayBoard function. In this function, I had to calculate the exact position of each tile (or number) and display it either forward or backwards, since the board “snakes” around. To achieve this, I checked if each row was an odd or even row, and counted backwards if it was odd. I also was forced to calculate the spacing between each column of the board, which I did by subtracting the length of the current number by 8 and then adding spaces for each missing number.

Possible Improvements

In the future, I could modify the Game class to support an infinite amount of players. I could also display the board with the current player positions after every move, instead of just once at the beginning of the game. This would be a significant undertaking, however, as I would have to account for much more in the displayBoard function.